

Design and train LSTM model for real world application of your choice

```
import numpy as np
from keras.datasets import imdb
from keras.models import Sequential
from keras.layers import Embedding, LSTM, Dense, Dropout
from keras.preprocessing.sequence import pad_sequences
```

```
vocab_size = 10000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)
```

```
maxlen = 200
X_train = pad_sequences(X_train, maxlen=maxlen)
X_test = pad_sequences(X_test, maxlen=maxlen)
```

```
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=128),
    LSTM(64),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.build(input_shape=(None, maxlen))
model.summary()
```

Model: "sequential_7"

Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	(None, 200, 128)	1,280,000
lstm_7 (LSTM)	(None, 64)	49,408
dropout_7 (Dropout)	(None, 64)	0
dense_7 (Dense)	(None, 1)	65

Total params: 1,329,473 (5.07 MB)
 Trainable params: 1,329,473 (5.07 MB)
 Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=3, batch_size=64, validation_split=0.2)
```

```
Epoch 1/3
313/313 ————— 89s 276ms/step - accuracy: 0.7020 - loss: 0.5594 - val_accuracy: 0.8500 - val_loss: 0.3631
Epoch 2/3
313/313 ————— 140s 269ms/step - accuracy: 0.9033 - loss: 0.2586 - val_accuracy: 0.8626 - val_loss: 0.3228
Epoch 3/3
313/313 ————— 148s 288ms/step - accuracy: 0.9336 - loss: 0.1871 - val_accuracy: 0.8642 - val_loss: 0.3350
<keras.src.callbacks.history.History at 0x7a54138a4150>
```

```
# history = model.fit(
#     X_train, y_train,
#     epochs=5,
#     batch_size=64,
#     validation_split=0.2
# )
```

```
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {accuracy:.4f}')
```

```
782/782 ————— 28s 36ms/step - accuracy: 0.8619 - loss: 0.3491
Test Accuracy: 0.8628
```

```
word_index = imdb.get_word_index()
reverse_word_index = {value: key for key, value in word_index.items()}

def decode_review(encoded_review):
    return ' '.join([reverse_word_index.get(i - 3, '?') for i in encoded_review])
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 ————— 1s 0us/step
```

```
import random

random_idx = random.randint(0, len(X_test) - 1)
print(f"Review index: {random_idx}")
```

```
print(decode_review(X_test[random_idx]))
prediction = model.predict(X_test[random_idx:random_idx+1])[0][0]
sentiment = "Positive" if prediction > 0.5 else "Negative"
print(f"Prediction: {sentiment} ({prediction:.2f})")
```

↻ Review index: 6101
becomes the main focus the episode falls apart part is an ? stereotype and his flat performance highlights this flaw his exploitati
1/1 ————— 0s 51ms/step
Prediction: Negative (0.39)

